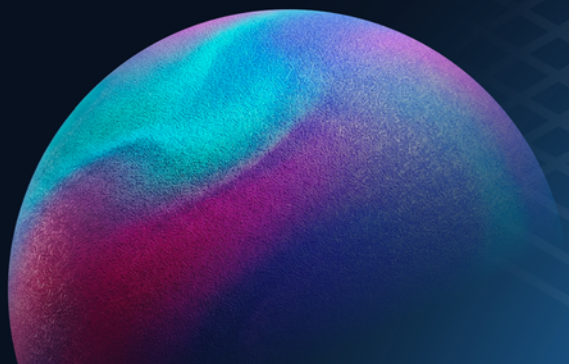
BSOS

智能合約安全檢測報告

檢測工具 Veros

檢測日期 2026-09-08



AI 草稿 — 未經人工複核，不可作為交付文件

本報告的分類判斷中有 21 筆由模型產出、尚未經人工確認。分類判斷是本報告可信度的來源，必須逐筆複核後才可交付；複核完成請以 veros confirm 標記，再重新產出報告。

目錄

- 檢測範圍與方法
- 摘要
- 檢測結果

---

isle-enterprise-contract 智能合約安全檢測報告

檢測範圍與方法

掃描範圍

本次檢測涵蓋以下 33 個 Solidity 原始檔（合計 4557 行）。此清單即本報告的效力邊界 —— 未列於此的檔案不在本次檢測範圍內。

檔案	行數
contracts/IsleGlobals.sol	125
contracts/LoanManager.sol	985
contracts/LoanManagerStorage.sol	27
contracts/Pool.sol	372
contracts/PoolAddressesProvider.sol	204
contracts/PoolConfigurator.sol	415
contracts/PoolConfiguratorStorage.sol	21
contracts/Receivable.sol	119
contracts/ReceivableStorage.sol	20
contracts/WithdrawalManager.sol	483
contracts/WithdrawalManagerStorage.sol	15
contracts/abstracts/Governable.sol	65
contracts/interfaces/IGovernable.sol	55
contracts/interfaces/IIsleGlobals.sol	118
contracts/interfaces/IIsleGlobalsEvents.sol	52
contracts/interfaces/ILoanManager.sol	115
contracts/interfaces/ILoanManagerEvents.sol	87
contracts/interfaces/ILoanManagerStorage.sol	49
contracts/interfaces/IPool.sol	92
contracts/interfaces/IPoolAddressesProvider.sol	121
contracts/interfaces/IPoolConfigurator.sol	172
contracts/interfaces/IPoolConfiguratorEvents.sol	77
contracts/interfaces/IPoolConfiguratorStorage.sol	34
contracts/interfaces/IReceivable.sol	37
contracts/interfaces/IReceivableEvent.sol	27
contracts/interfaces/IWithdrawalManager.sol	165
contracts/interfaces/IWithdrawalManagerStorage.sol	23
contracts/libraries/Errors.sol	198
contracts/libraries/PoolDeployer.sol	21
contracts/libraries/ReentrancyGuard.sol	40
contracts/libraries/types/DataTypes.sol	104
contracts/libraries/upgradability/UUPSProxy.sol	8
contracts/libraries/upgradability/VersionedInitializable.sol	111

檔案內容雜湊（SHA-256），供比對交付程式碼是否與受檢版本一致：

contracts/IsleGlobals.sol — 3987b303f8f62cd4848bb4a6bf2bad8a090fd51cbb0e12b0370e157d087da756

contracts/LoanManager.sol — a4e943a32782f618a5473feb75732a04430cfe6a48e7f0fb118fb4847fe3c966

contracts/LoanManagerStorage.sol — 4cee1e6a56a2928591a69b1ef75c7552ae262d149bce39e14259838b750fbca7

contracts/Pool.sol — 3ebb28118848e0b390ebe88568795a9fc4e0cb88c0c958695a692680664a5d97

isle-enterprise-contract 智能合約安全檢測報告

contracts/PoolAddressesProvider.sol — fcb12e5f417360315c0efde4768b00a8a7c0e4052fb78e0ad6030b477e7fc055

contracts/PoolConfigurator.sol — 72c5c0db996978e18d8cb4e0ea477bbdc288930beac5a6211a602240709bc9b1

contracts/PoolConfiguratorStorage.sol — 4661a48cedc7755a90f4fdd603d9e0cec182e06b0120d93c68af0859665cc7b2

contracts/Receivable.sol — 3326392b70ffe4b8916b94d538a26c2be3b8fdf954e0468e04abbb127010c914

contracts/ReceivableStorage.sol — e066b3e0f9a11cc9601cf769bc201e39b85f66a043e5ff8c02025317df4d21fa

contracts/WithdrawalManager.sol — 7b8662684f73407e0687bac609e9df284cdfc42057d19520e72e00f7c8469d34

contracts/WithdrawalManagerStorage.sol — 72d5f9a256509d765cb50886e74729e40283f9e215376d015d8d2d156d0ccee2

contracts/abstracts/Governable.sol — c23132fae93d3293dc45cfef0bc5c7b214d130a7457c33ad628208fb47ef7f3f

contracts/interfaces/IGovernable.sol — 15e42654c06eba75b5fd08c150e91fbfc93e3141da950cf8b827acaaca60a6bc

contracts/interfaces/IIsleGlobals.sol — 3c18df7a4840be63b81d4ba30f50be6a3e6b3b65e6b943bc723dc43309ea7080

contracts/interfaces/IIsleGlobalsEvents.sol — b839a4dd2428e859daadf3189d61bbb7824ed3587de3930b9b6fd15d32dcf3e4

contracts/interfaces/ILoanManager.sol — 516bd951dad4523cc19f3721ce798603bb86d6869ea4140430b6dc5df82fad49

contracts/interfaces/ILoanManagerEvents.sol — fd61b350a2b3237d3b4c57c711cda24b2a12ca680bd4960e3e9ab367918814a0

contracts/interfaces/ILoanManagerStorage.sol — 62f152636716b1f29bf056d50e99fe4366d1100a65b2203be25947d0e734acc6

contracts/interfaces/IPool.sol — 80df7b35418623f35460a31d87ebc3c5555cafc0799a05be4fd23e821ebab5f1

contracts/interfaces/IPoolAddressesProvider.sol — c16f2a9a3fe1db98bca4801d0b89d8e38358c8e7c63d2e869453237b2d1cae04

contracts/interfaces/IPoolConfigurator.sol — 96da95b1e7aced3884198db2251b68e7b7cb77816bc2dc3e0e884ad2845ab9f7

contracts/interfaces/IPoolConfiguratorEvents.sol — 4174280c1d83cdacdf042889fffef2fd033edc611e8afae0249b6ba7999e6aaf

contracts/interfaces/IPoolConfiguratorStorage.sol — c1e231d1db62fb9e9a3e037a3abadde839e4bf09289c8161c1fb1bff055a0558

contracts/interfaces/IReceivable.sol — 1f59172b4c1158f8331f990a6f76330f687cfe2f2545b56d20dfcd0e661e016a

contracts/interfaces/IReceivableEvent.sol — 4c5af0d433d5182fb9e6022f3ae1cb36f5c6198cc9f004df1b421baa5abcd10c

contracts/interfaces/IWithdrawalManager.sol — 0c52b08f5193399d0a88d589536a04a5fca271cdddf8667f465197b7858cba38

contracts/interfaces/IWithdrawalManagerStorage.sol — 4feb91d731a3524d37715be4db469a45895a7d8919aaaeb5ad3f80cd2da091c4

contracts/libraries/Errors.sol — fdc920a2a5ea9dbc0f7320e33c8cdecb7c786803028ed76b83ef6ee1eb2f2151

contracts/libraries/PoolDeployer.sol — ecff7ef85239a5ee36c8da5d3e8be4d8f572d1f2ef7ba8a0ae8218b36e742588

contracts/libraries/ReentrancyGuard.sol — c2fc2d7b7d40a86c119a054c373f3ce1d97d2c2ccffc34a9e495e1219cb5ed99

contracts/libraries/types/DataTypes.sol — 1be0d6fb2c6ff2de26849246bc2d5b6398bf5918b47c59589a03eb5968983095

contracts/libraries/upgradability/UUPSProxy.sol — 080abfe3e2d23e9c5447d5dea5dc0f7be5775603482a15526138571630af6679

contracts/libraries/upgradability/VersionedInitializable.sol — 5c4636c308adb1c0b0f4abcc0888471976a53f8f8dacf294a8640c945a805631

檢測方法

本次檢測依以下步驟執行：

1. **靜態分析掃描**：以 Slither 對「掃描範圍」所列全部原始檔執行完整 detector 掃描。
2. **逐筆判讀與補充比對**：對掃描產出的每一筆發現判定其處置分類並記錄判斷依據，工具回報的嚴重度與本報告呈現的嚴重度若有落差則逐筆附上調整理由；同時對範圍內每一份合約，逐條比對內部維護的邏輯漏洞情境庫（權限檢查實作、未保護的狀態變更、旗標未落實、價格源可操縱、記帳與實際結果脫鉤、簽章雜湊綁定範圍、可組合模組的交互失效等），並依受檢系統所屬業務領域比對該領域公開已知的事故模式，補靜態規則無法涵蓋的業務邏輯層級問題。
3. **產出與覆核**：彙整為本報告，並對共用同一判斷理由的發現群組進行抽查。

本次掃描的工具版本：Slither 0.11.4、solc 0.8.24、Foundry 1.7.1。重新掃描時使用相同版本，方能與本報告

的數字逐筆對照。

範圍限制

本報告由本檢測工具產出，解讀時請留意以下範圍限制：

- **偵測範圍**：靜態分析擅長偵測「程式寫法特徵層級」的問題（如重入模式、tx.origin 授權、弱亂數來源、未檢查的低階呼叫回傳值等）。
- **已知偵測邊界**：業務邏輯層級的問題 —— 例如權限檢查的實作邏輯錯誤、應存在而未實作的保護、經濟模型層面的攻擊（搶跑、滑點）—— 靜態規則無法窮舉，本工具以情境庫逐合約比對補充涵蓋，但其涵蓋程度不等同於系統性審計。
- **文件性質**：本報告為交付前之**自我檢查證明**，證明工程團隊已執行掃描並對每一筆發現完成逐筆判讀；其不構成、亦不取代由獨立第三方執行之完整安全審計。

檢測環境偏離揭露

1. 本次為針對 PR #90 的重新掃描

本次掃描的標的是 PR #90（fix/loanmanager-interest-accounting，commit c79b21f），即上一輪報告 [H-1]／[M-4] 兩項利息帳務缺陷修復後的程式碼。合約層的改動只有 contracts/LoanManager.sol 的兩處（accruedInterest() 恢復到期日上限、_advanceGlobalPaymentAccounting() 補回 domainStart 寫回），其餘為註解與測試。

分類沿用上一輪結果：105 筆掃描發現中 84 筆逐筆比對後沿用，20 筆因 LoanManager.sol 行號位移（+11）而重新對位並逐筆重新確認，1 筆為工具自動判定的命名風格項。上一輪的 11 筆人工發現中，[H-1] 與 [M-4]（原 ISL-105／ISL-106）經本次逐行複核確認已修復，依規則自人工發現移除；另新增 1 筆（既有池的存量沖銷與遷移入口缺口），詳見報告的發現章節。

2. Slither 無法直接解析本專案原始碼（沿續上一輪）

靜態分析工具 Slither 0.11.4 **無法直接解析本專案原始碼**，錯誤為 Type not found struct WithdrawalManager.CycleConfig。根因是 contracts/libraries/types/DataTypes.sol 內的四個 library（PoolConfigurator、WithdrawalManager、Receivable、Loan）與同名 contract 衝突，工具的名稱解析會解到 contract 上。

為完成本次掃描，檢測方在本機**暫時**將這四個 library 更名為 *_Types，並同步更新 contracts/ 底下 10 個引用處的 import 與型別引用，掃描結束後立即還原；本次交付不含任何合約改動。此改名為純識別字重新命名，不改變任何邏輯、控制流或儲存佈局，**每個檔案的行數與每一行的行號都未改變（逐檔確認新增行數等於刪除行數），因此本報告引用的所有**「檔案:行號」對原始碼仍然成立。副作用有兩處：一是「掃描範圍」章節記錄的檔案雜湊值中，有 11 個檔案是改名後的值，與原始碼不符；二是工具輸出的函式簽章中，型別名以改名後的形式出現（Loan_Types.PaymentInfo、Receivable_Types.Create 等），原始碼中的對應名稱為 Loan.PaymentInfo、Receivable.Create。兩者皆不影響發現本身的內容與行號。

補充一點上一輪未記錄的操作細節：上一輪的說明是「以 import alias 接回原名」，實測在

WithdrawalManagerStorage.sol 與 IWithdrawalManager.sol 這類「alias 名稱本身就等於衝突 contract 名」的情況下無效——Slither 仍會把該識別字解析到 contract 上。本次改為在引用處直接使用 *_Types 名稱，才使掃描成功。

建議儘速修正此命名衝突。 在修正之前，本專案無法被 Slither 掃描 —— 若 CI 中已配置靜態分析，它會回報「零發現」而非失敗，等同於靜態分析形同虛設。本次首輪掃描即出現此現象：工具回傳 success: false 但流程仍以「無發現」結束。

isle-enterprise-contract 智能合約安全檢測報告

摘要

受檢對象：isle-enterprise-contract
檢測期間：2026-09-08T15:07:41.415227
受檢版本：c79b21faf65e8d9766255d3102b4399be0264c5f

本次共提出 115 項發現，依嚴重程度分布如下：

嚴重程度	筆數
Critical	0
High	4
Medium	26
Low	54
Informational	31
總計	**115**

揭露：上述筆數中，15 筆由風格預分類自動判定（naming-convention 12 筆、unindexed-event-address 3 筆），非人工逐筆判讀；84 筆沿用前次掃描的分類結果。

全部發現逐筆列於「檢測結果」，依處置分類分組。

協定概要

Isle v1 是應收帳款融資協定：買方（borrower）以短天期應收帳款憑證向資金池借款，存款人（lender）提供資金賺取利息。程式碼衍生自 Maple v2，為逐檔手工改寫而非 fork。

本次檢測範圍為 contracts/ 目錄下 33 個 Solidity 檔案、共 4,557 行，不含 modules/ 底下的相依套件（OpenZeppelin v4.9.2、forge-std、prb-math）。

核心合約分工：

合約	職責
Pool	ERC-4626 份額金庫，持有閒置流動性，負責存款與贖回的份額換算
PoolConfigurator	市場層設定與資金調度樞紐，持有第一損失準備（pool cover）
LoanManager	貸款生命週期與利息帳務，撥款後短暫持有資金
WithdrawalManager	以「週期 + 視窗」排隊處理贖回，持有排隊中的份額憑證
Receivable	應收帳款憑證（ERC-721）
IsleGlobals	全域白名單、協定費率、暫停開關
PoolAddressesProvider	各核心合約的位址註冊與代理升級入口

違約時的損失吸收順序

本協定承作的是**無實體擔保**的短天期放款，借款人的償付能力由 pool admin 於鏈下審核。合約層不存在擔保率、清算價格或自動平倉機制；貸款逾期後的處理路徑為「認列減值 → 觸發違約 → 動用第一損失準備」，不足部分由存款人承擔。這是本類協定的共同設計取捨，非程式缺陷，但它決定了本報告中與「特權角色參數無上下限」相關的發現為何重要 —— 在沒有自動化風控的系統裡，人工設定的參數就是

最後一道防線。

資產保管與流向

系統流通兩種資產：**pool asset**（ERC-20，實務上為 USDC，由 governor 白名單控管）與 **receivable**（ERC-721 應收帳款憑證）。

pool asset 在生命週期中會停留在四個位置：

停放位置	何時有餘額	誰能移動
Pool	存款人存入後、尚未放款前的閒置流動性	PoolConfigurator（撥款）與 Pool 自身（贖回）
LoanManager	撥款後、借款人提領前；以及還款進出的同一筆交易內	持有對應 receivable 憑證的人；合約自身的分配邏輯
PoolConfigurator	第一損失準備（pool cover）	pool admin
借款人指定地址	提領之後	已離開協定

兩點值得存款人注意：

1. Pool 在部署時即對 PoolConfigurator 授出**無上限且不可撤銷**的 ERC-20 授權。因此「誰能動資金池裡的錢」實際上等於「誰被註冊為 loan manager」，而該註冊由 governor 控制。
2. depositCover 開放任何人存入第一損失準備，但只有 pool admin 能提領。這是單向的資金投入。

receivable 憑證的流向為：建立後鑄造給賣方 → 賣方以憑證換取資金（憑證轉入 LoanManager）→ 買方清償後銷毀。憑證的內容（買方、賣方、面額、到期日）在建立時由呼叫者自填，合約端不做真實性驗證。

角色與權限

系統有兩個具名特權角色。

Governor

單一位址，透過提名／接受兩段式移轉。合約層不強制多簽或時間鎖。權限涵蓋：

- 替換任一核心合約的實作（升級），或直接改寫核心合約的註冊位址
- 增刪全部白名單（可用資產、可用憑證、pool admin 名單）
- 設定協定費率、全域與單一函式的暫停開關
- 指派與更換 pool admin

Governor 同時具備 pool admin 的全部權限。**這個角色的權限邊界涵蓋資金池的全部資產**，其私鑰管理方式（多簽門檻、時間鎖、金鑰保管）屬於部署與營運層的控制，不在程式碼檢測範圍內，建議另行向存款人揭露。

Pool Admin

由 governor 指派，且必須在白名單內。權限涵蓋：

- **決定是否撥款** —— 這是唯一的信用審核關卡；合約層不對額度、利率、天期做任何合理性檢查
- 提領第一損失準備（受最低準備門檻限制）
- 設定管理費率、贖回週期參數、交易對手白名單、是否開放公眾存款
- 認列與撤銷貸款減值、觸發違約

無權限函式

以下函式任何位址皆可呼叫：建立應收帳款憑證、存入第一損失準備、推進利息帳務、代為還款。
存款與贖回受白名單或公開開關限制。

檢測結果

本次共 115 項發現，本章逐筆列出其中 15 項。其餘 100 項為經查證為誤報者，以及低嚴重度且經判定為可接受風險者，逐筆紀錄保留於工作底稿，可依需要調閱。

嚴重度	筆數
Critical	0
High	0
Medium	13
Low	1
Informational	1

處置分類	筆數
已確認需修復（A）	11
已知風險但可接受（B）	4

編號 [H-1] 為嚴重度代碼（C 危急／H 高／M 中／L 低／I 資訊）加該嚴重度內的序號，僅指派給經判定確實成立、需要處置的發現；經查證為誤報或已接受之風險沿用掃描編號。

已確認需修復（A）

[M-1] repayLoan 缺少 nonReentrant，本金在分配與扣帳之間被重複計入份額價格

位置：contracts/LoanManager.sol:263-300

嚴重度異動（掃描工具判定為 High）：工具判 High 係假設 asset 為任意 ERC20。本專案的 asset 受 IsleGlobals.isPoolAsset 白名單控管（PoolConfigurator.sol:99），目前部署為無 transfer hook 的 USDC，另兩個取得控制權的地址（poolAdmin、isleVault）皆為協定自有。故實際可利用性需要額外的治理層失誤配合，降為 Medium；但因為修法成本極低且同檔其他函式已有防護，仍列為必須修復。

說明：repayLoan（LoanManager.sol:263-300）的八個步驟中，步驟 4 _distributeClaimedFunds（:280 → :870-893）先把本金與淨利息轉進 pool、再分別轉出 adminFee 與 protocolFee，步驟 5（:283-285）才遞減 principalOut。三筆 safeTransfer 都是對外部地址的呼叫，而整個函式沒有重入鎖 —— 同檔的 fundLoan、removeLoanImpairment 都有。

影響：在步驟 4 與步驟 5 之間，principalOut 尚未遞減，但本金已經進入 pool 餘額，PoolConfigurator._totalAssets()（PoolConfigurator.sol:344）＝ pool 餘額 + AUM 會把同一筆本金計入兩次，份額價格在該視窗內虛高。若攻擊者能在此視窗取得執行權（asset 具 transfer hook，或 poolAdmin／isleVault 為合約），即可在虛高價格下贖回既有份額，差額由其餘存款人承擔。實測的虛增幅度為 100,986,301,368（約 $100,986.30，等於本金加該期利息），在該視窗內以此價格贖回即可取走不屬於自己的資產。

PoC：已以可執行測試實證（tests/integration/concrete/loan-manager/poc/AuditFindings.t.sol，7 passed）：
(1) test_M1_ReentrantRepayLoanIsNotBlockedByAnyGuard —— 在 repayLoan 執行中重入 repayLoan，第二次進入**沒有**被 Errors.ReentrancyGuardReentrantCall() 擋下（實測 revert selector 為 0x4e487b71，即

Panic，代表它已通過入口、執行到深處的帳務運算才失敗），證明本函式確實沒有重入鎖。
(2) test_M1_TotalAssetsDoubleCountsPrincipalDuringRepay —— 在同一時點讀 Pool.totalAssets()：呼叫前 1,500,986,301,368、**呼叫中 1,601,972,602,737**、呼叫後 1,500,986,301,369，期間虛增 100,986,301,368（USDC 6 位小數，約 $100,986.30）。量級等於本金 $100,000 加該期利息 $986.30 —— 此時池內已收到本金與淨利息，AUM 側的 principalOut 與 accountedInterest 卻都還沒沖銷，兩條腿都被重複計入。
成立條件（測試中如何構造，也就是實際部署要避免的組合）：pool asset 必須是會把控制權交給收款方的代幣（ERC777／ERC1363 類的 recipient hook），**且**該收款方是有程式碼的合約。測試以 vm.etch 把帶 hook 的 ERC20 覆蓋到既有 USDC mock 上，並把 isleGlobals.isleVault 設為一個合約 —— 注意取得控制權的是**協定自己的手續費收款地址**，不需要任何額度、份額或特權。現行部署的 asset 為標準 USDC（無 hook），故目前不可觸發。

建議修法：在 repayLoan（LoanManager.sol:263）加上 nonReentrant modifier，與 fundLoan（:242）、removeLoanImpairment（:379）一致。另建議把步驟 5 的 principalOut 遞減（:283-285）移到步驟 4 的資金分配（:280）之前，讓 _totalAssets() 在整筆交易期間都不會重複計入本金。

[M-2] repayLoan 缺少 nonReentrant（Slither 對同一函式的第二筆回報）

位置：contracts/LoanManager.sol:263-300

嚴重度異動（掃描工具判定為 High）：同 [M-1]：同一函式、同一修法，降級理由一致。

說明：與 [M-1] 同一根因：repayLoan（LoanManager.sol:263-300）無重入鎖，外部呼叫（:275 的 safeTransferFrom 與 :280 的 _distributeClaimedFunds）之後仍有狀態寫入。Slither 依受影響的狀態變數分開回報。

影響：同 [M-1]：principalOut（:284）與 accountedInterest（:290 的 _handlePaymentAccounting）在外部呼叫之後才更新，期間 PoolConfigurator._totalAssets() 讀到不一致的中間狀態，份額價格短暫虛高。實測的虛增幅度為 100,986,301,368（約 $100,986.30，等於本金加該期利息），在該視窗內以此價格贖回即可取走不屬於自己的資產。

PoC：已以可執行測試實證（tests/integration/concrete/loan-manager/poc/AuditFindings.t.sol，7 passed）：
(1) test_M1_ReentrantRepayLoanIsNotBlockedByAnyGuard —— 在 repayLoan 執行中重入 repayLoan，第二次進入**沒有**被 Errors.ReentrancyGuardReentrantCall() 擋下（實測 revert selector 為 0x4e487b71，即 Panic，代表它已通過入口、執行到深處的帳務運算才失敗），證明本函式確實沒有重入鎖。
(2) test_M1_TotalAssetsDoubleCountsPrincipalDuringRepay —— 在同一時點讀 Pool.totalAssets()：呼叫前 1,500,986,301,368、**呼叫中 1,601,972,602,737**、呼叫後 1,500,986,301,369，期間虛增 100,986,301,368（USDC 6 位小數，約 $100,986.30）。量級等於本金 $100,000 加該期利息 $986.30 —— 此時池內已收到本金與淨利息，AUM 側的 principalOut 與 accountedInterest 卻都還沒沖銷，兩條腿都被重複計入。
成立條件（測試中如何構造，也就是實際部署要避免的組合）：pool asset 必須是會把控制權交給收款方的代幣（ERC777／ERC1363 類的 recipient hook），**且**該收款方是有程式碼的合約。測試以 vm.etch 把帶 hook 的 ERC20 覆蓋到既有 USDC mock 上，並把 isleGlobals.isleVault 設為一個合約 —— 注意取得控制權的是**協定自己的手續費收款地址**，不需要任何額度、份額或特權。現行部署的 asset 為標準 USDC（無 hook），故目前不可觸發。

建議修法：同 [M-1]，單一 nonReentrant modifier 即同時涵蓋兩筆。

[M-3] _handleCover 以原生 transfer 動用第一損失準備，回傳值未檢查

位置：contracts/PoolConfigurator.sol:382-392

嚴重度異動（掃描工具判定為 High）：工具判 High 係假設代幣行為未知。目前部署資產為標準回傳 true 的 USDC，兩條失敗路徑都需要 governor 先把非標準代幣加進 isPoolAsset 白名單才會觸發，不是任何外部人可直接觸發的資金損失，故降為 Medium。但這是一行的修法，且缺陷會同時影響資金正確性與違約流程的可用性，仍列為必須修復。

說明：_handleCover（PoolConfigurator.sol:389）在觸發違約時把第一損失準備（pool cover）撥給 pool，用的是原生 transfer 且忽略回傳值。同一份合約其餘所有 asset 操作都走 SafeERC20（:185、:247、:256），此處是明顯遺漏。asset 是 governor 白名單的任意 ERC20，不保證失敗時 revert。

影響：兩種常見代幣各自造成不同後果：(a) 回傳 false 而不 revert 的代幣 —— poolCover -= coverAmount_（:387）已先扣減，帳面準備金減少但資金並未轉出，差額無法回復，等同憑空蒸發一筆準備金；(b) 完全不回傳資料的代幣（USDT 型）—— OZ 的 IERC20.transfer 解碼空回傳值會 revert，使 triggerDefault 全面卡死，違約無法認列，損失無法從準備金吸收。後者發生的時機正是池子最需要違約流程的時候。

PoC：不需要攻擊者，由代幣選擇決定：governor 將白名單納入回傳 false 型或無回傳值型的 ERC20 作為 pool asset，隨後任一筆貸款走到 triggerDefault → _handleCover，即分別重現 (a) 準備金帳實不符或 (b) 違約流程 revert。以 USDC 部署時不觸發。

建議修法：改用 IERC20(asset).safeTransfer(pool, coverAmount_)（本檔已 using SafeERC20 for IERC20，無需新增 import）。並建議把 poolCover -= coverAmount_ 移到轉帳成功之後。

[M-4] protocolFee + adminFee 合計無上限，可使 fundLoan 永久 revert

位置：contracts/LoanManager.sol:770-775

說明：_queuePayment（LoanManager.sol:770-775）取 feeRate_ = protocolFee_ + adminFee_ 後呼叫 _getNetInterest(interest_, feeRate_)，其實作為 interest_ * (HUNDRED_PERCENT - feeRate_) / HUNDRED_PERCENT（:680）。兩個費率的 setter —— IsleGlobals.setProtocolFee（IsleGlobals.sol:90，onlyGovernor）與 PoolConfigurator.setAdminFee（PoolConfigurator.sol:144，onlyAdminOrGovernor）—— 都沒有任何上下界檢查。uint24 上限 16,777,215 相對於 HUNDRED_PERCENT = 1e6 等於 1677%，DataTypes.sol:9 的註解「uint24 adminFee; max = 1.6e7 (1600%)」顯示開發者已意識到型別容許超過 100%，但未加檢查。

影響：任一方或兩方合計超過 100% 時，HUNDRED_PERCENT - feeRate_ 在 Solidity 0.8.19 下溢 revert，fundLoan 全面失效直到費率被調回。既有 payment 不受影響（費率在建立時快照，超過 100% 的組合根本無法建立成功），故不影響還款路徑，但新撥款完全癱瘓：借款人無法取得資金，池內閒置流動性無法產生收益，且在費率被察覺並調回之前無法自行恢復。

PoC：已以可執行測試實證（tests/integration/concrete/loan-manager/poc/AuditFindings.t.sol::test_M4_CombinedFeeAboveHundredPercentBricksFundLoan）：governor 呼叫 isleGlobals.setProtocolFee(600_000)（60%）、pool

admin 呼叫 poolConfigurator.setAdminFee(500_000)（50%），兩支 setter **各自都接受**（實測寫入成功，兩者皆為 uint24 且無任何邊界檢查），合計 110% > HUNDRED_PERCENT = 1e6。隨後任一筆 fundLoan 必定 revert：_queuePayment（LoanManager.sol:790）→ _getNetInterest（:695）計算 interest_ * (HUNDRED_PERCENT - feeRate_) / HUNDRED_PERCENT，在 0.8.19 下減法下溢為 Panic 0x11，測試以 vm.expectRevert(stdError.arithmeticError) 驗證。撥款功能永久停擺，直到有人把費率調回來。

建議修法：在兩個 setter 加上界檢查，且必須檢查**合計**：setAdminFee 內驗證 adminFee_ + globals.protocolFee() <= HUNDRED_PERCENT，setProtocolFee 內同理；或在 _queuePayment 以 _min(feeRate_, HUNDRED_PERCENT) 兜底。建議兩者都做，並把合理上限（例如 50%）寫成常數。

[M-5] maxCoverLiquidation 無上限，可使 triggerDefault 永久 revert

位置：contracts/PoolConfigurator.sol:382-392

說明：_handleCover（PoolConfigurator.sol:382-392）計算可動用的第一損失準備 availableCover_ = poolCover * _config.maxCoverLiquidation / HUNDRED_PERCENT。setMaxCoverLiquidation 沒有上限檢查，該值可被設為超過 100%，使 availableCover_ 大於實際持有的 poolCover；後續扣減時下溢 revert。

影響：triggerDefault 整體不可用 —— 違約無法認列，損失無法從準備金吸收，逾期部位持續掛在帳上並繼續以原 issuance rate 計息，進一步扭曲資產淨值。觸發條件為 maxCoverLiquidation > 1e6 且 losses_ > poolCover，後者在無擔保信貸池是常態（postmortem 記載 53 筆已還款中 52 筆逾期）。失效發生的時機正是最需要違約流程的時候。

PoC：不需要攻擊者，由設定失誤觸發：governor 或 pool admin 呼叫 setMaxCoverLiquidation 傳入大於 HUNDRED_PERCENT（1e6）的值，例如誤把 100% 寫成 100×1e6。隨後任一筆損失大於 poolCover 的貸款呼叫 triggerDefault，即在 _handleCover 下溢 revert。

建議修法：在 setMaxCoverLiquidation 加 if (maxCoverLiquidation_ > HUNDRED_PERCENT) revert ...；並在 _handleCover 以 _min(availableCover_, poolCover) 兜底。

[M-6] Receivable.createReceivable 完全無存取控制

位置：contracts/Receivable.sol:59-76

說明：Receivable.createReceivable（Receivable.sol:59-76）鑄造代表一張應收帳款的 ERC-721 憑證，函式上沒有任何 modifier —— 任何地址都可以指定買方、賣方、面額與到期日鑄出一張憑證。合約層面沒有任何機制驗證這張憑證背後真的存在一筆應收帳款。

影響：放款路徑本身沒有被打穿（見 PoC），因此不會直接失血。實際影響有三：(a) 任何人可把憑證塞進任意賣方錢包，污染以此 NFT 為準的鏈下對帳與報表；(b) _tokenIdCounter 可被無成本灌大；(c) 這是「鏈下事實無法被合約驗證」在本專案最直接的體現 —— 「一張應收帳款存在」這個宣稱在鏈上不需要任何憑據，而整個放款決策建立在這個宣稱之上。

PoC：任意地址直接呼叫 createReceivable(buyer, seller, faceAmount, repaymentTimestamp, currencyCode) 即鑄出憑證，無需任

何權限。攻擊者無法據此自助取得貸款：已逐行確認 LoanManager.requestLoan 要求 msg.sender == receivableInfo_.buyer（:194、:943-947），_revertIfInvalidReceivable（:949-965）另外驗證 poolConfigurator_.buyer() == buyer_ 與 isSeller(seller_)，撥款還需 onlyPoolAdmin。故可利用面限於上述三項污染效果。

建議修法：把 createReceivable 限制為白名單角色（governor、pool admin，或 isSeller 名單），或改為需要買方簽章授權。長期建議引入 attestation 機制讓外部可驗證憑證對應真實發票。

[M-7] 借款人自訂的 gracePeriod 無上限，可使 triggerDefault 永久無法觸發

位置：contracts/LoanManager.sol:173-228

說明：requestLoan（LoanManager.sol:173-228）由借款人自行提交貸款參數，其中包含 gracePeriod（寬限期）。這個參數的用途是保護出借方 —— triggerDefault 必須等到期日加上寬限期之後才能觸發。由借款人決定一個用來限制出借方何時能宣告其違約的參數，方向本身就是反的，而合約對其上限沒有任何檢查。

影響：借款人提交一個極大的 gracePeriod（例如 2^255），該筆貸款即永遠無法被觸發違約：本金無法透過違約流程回收，第一損失準備無法動用，該部位永久掛在 principalOut 上並持續計息，使資產淨值長期虛增。存款人面對的是一筆帳面存在、實質永遠無法處置的債權。

PoC：已以可執行測試實證（tests/integration/concrete/loan-manager/poc/AuditFindings.t.sol，兩支）：(1) test_M7_BorrowerChosenGracePeriodBlocksDefaultForever —— 借款人（requestLoan 的呼叫者本人，受 _revertIfCallerNotReceivableBuyer 限定為 receivable 的 buyer）傳入 gracePeriod = 36_500 days（一百年），requestLoan（LoanManager.sol:184-239）**沒有任何邊界檢查**，原值寫進 Loan.Info.gracePeriod（實測 getLoanInfo().gracePeriod 等於傳入值）。撥款後推進到到期日 +3650 天，poolConfigurator.triggerDefault 仍因 block.timestamp <= dueDate + gracePeriod（:438）而 revert。
(2) test_M7_MaxGracePeriodOverflowsTheDefaultCheck —— 傳入 type(uint256).max 時，同一行的 dueDate + gracePeriod 直接算術溢位，triggerDefault 以 Panic 0x11 revert。兩種寫法都讓該筆貸款**永遠無法被認列違約**，第一損失準備因此無法動用。

建議修法：在 requestLoan 對 gracePeriod_ 設合理上限（例如 90 days），對 rates_ 兩個元素設上下限；或改為由 pool admin 在 fundLoan 時指定這三個參數，讓借款人只能提出申請、不能決定保護條款。

[M-8] setExitConfig 的 cycleDuration 無上限，pool admin 可實質凍結全部贖回

位置：contracts/WithdrawalManager.sol:105-158

說明：setExitConfig（WithdrawalManager.sol:105-158）設定贖回的週期長度 cycleDuration，贖回必須落在對應週期的視窗內才能執行。這個參數沒有上限檢查，pool admin 可將其設為極大值，使下一個贖回視窗落在極遠的未來。

影響：全體存款人的贖回被實質凍結，且無其他路徑取回底層資產 —— 已逐行確認 Pool.withdraw 直接 revert Pool_WithdrawalNotImplemented（:193），redeem 必經 maxRedeem → isInExitWindow（PoolConfigurator.sol:310-315）；removeShares 只能取回 pool share 憑證，換不回底層資產。此為單一角色即可執行的資金凍結。

PoC：pool admin 呼叫 setExitConfig 傳入極大的 cycleDuration_（例如 type(uint64).max）。既有與新增的贖回請求其視窗起點被推至極遠未來，所有 redeem 因 isInExitWindow 為假而 revert。此模式對應本次領域調查歸納的 D-CREDIT-03，詳見 audit/DOMAIN_RESEARCH.md 第 3 節。

建議修法：對 cycleDuration_ 設上限（例如 90 days）與下限；並考慮加入「設定變更不得延後既有 pending 請求的既定視窗」的保護。

[M-9] 升級的原子性完全依賴呼叫者傳入正確的 params，否則 initialize 對任何人開放

位置：contracts/libraries/upgradability/VersionedInitializable.sol:37-60

說明：VersionedInitializable（libraries/upgradability/VersionedInitializable.sol:37-60）以 revision > lastInitializedRevision 作為 initialize 的守門條件，函式本身不檢查呼叫者。升級的原子性完全依賴呼叫者在 setXxxImpl 傳入正確的 params —— 若傳入空的 params，implementation 被換上但 initialize 未被呼叫，該函式在新 revision 下對任何人開放。

影響：在升級與初始化之間的區塊窗口內，任何人可搶先呼叫 initialize 並填入自己選定的參數，取得該合約的控制權。目前**尚未可利用**：三份合約的 revision 皆為 0x1，代理上的 lastInitializedRevision 已是 1，條件為假，再次呼叫會 revert。風險在下一次升級 —— 而 postmortem 第 6.1 節第 2 點規劃的 ISL-105/106 修復正是一次 revision 升到 2 的升級，屆時此窗口會真實存在。這是「修復缺陷的那次操作本身帶著新風險」的典型情況，必須在執行該修復之前先處理。

PoC：條件：下一次升級時 setXxxImpl 以空的 params 呼叫（或以非原子方式先換 implementation 再另發交易 initialize）。攻擊者監看 mempool，於 implementation 更換的交易之後、初始化交易之前，自行呼叫該代理的 initialize 並傳入自選參數，即以新 revision 完成初始化並取得控制權。

建議修法：在三份合約的 initialize 加入呼叫者檢查（msg.sender == address(ADDRESSES_PROVIDER) 或 proxy admin），使搶跑不可行；並在 setXxxImpl 系列函式要求 params.length != 0，把「必須原子升級」從營運紀律變成合約強制。

[M-10] 既有池的 accountedInterest 存量虛增沒有任何沖銷路徑，且合約未提供升級遷移入口

位置：contracts/LoanManager.sol:29-632

說明：PR #90 修好了造成 accountedInterest 系統性虛增的兩個成因（_advanceGlobalPaymentAccounting 補回 domainStart 寫回，accruedInterest() 恢復到期日上限），但**沒有處理已經寫進 storage 的存量**。逐行確認 accountedInterest 的全部寫入點：只有 _advanceGlobalPaymentAccounting（LoanManager.sol:609）的遞增、_updateIssuanceParams（:620）的原值回寫、以及 _compareAndSubtractAccountedInterest（:632）的遞減 —— 而該遞減以 _min(accountedInterest, amount_) 為上界，每筆 payment 只沖銷自己那一份，永遠不會把歷史多記的部分沖掉。合約也沒有任何特權函式可以修正這個值。同時 LOAN_MANAGER_REVISION 仍為 0x1（:29），initialize 帶 initializer（:53），代表既有池升級後**無法再跑一次初始化**，合約層目前不存在可執行沖銷的遷移入口。

影響：對**新部署的池**沒有影響：accountedInterest 從 0 開始，修復後不再產生偏差。對**既有池**則是：升級只會停止繼續虛增，已累積的差額仍留在 accountedInterest → assetsUnderManagement()（:127）→ Pool.totalAssets() → 份額價格。存款人依此價格贖回仍會取走無對應債權的資產，先贖回者的超額由後贖回

者承擔。另一面是升級當下 accruedInterest() 的語意改變（逾期期間不再預先承認利息），份額價格會出現一次性下修，若未預先向存款人說明，會被誤讀為虧損事件。

PoC：已以可執行測試實證
（tests/integration/concrete/loan-manager/poc/AuditFindings.t.sol::test_M10_InjectedExcessSurvivesEverySettlementPath）：以 vm.store 直接把 5,000,000,000（$5,000）的虛增寫進 accountedInterest（slot 1，offset 0，uint112；同 slot 的 principalOut 已確認未受影響），模擬既有池已累積的存量。接著把池子走過**合約提供的每一條結算路徑**：updateAccounting() → impairLoan() → removeLoanImpairment() → repayLoan() → 再一次 updateAccounting()。貸款全數清償後 principalOut 歸零，帳面本應為平，實測 accountedInterest 仍為 **5,000,000,000**、assetsUnderManagement() 亦為 **5,000,000,000** —— 一分未沖。這證實 _compareAndSubtractAccountedInterest（:632）的減量上界只涵蓋各筆 payment 自己的份額，合約中不存在任何能把歷史多記金額寫回的路徑。另以靜態確認 LOAN_MANAGER_REVISION 仍為 0x1（:29）、initialize 帶 initializer（:53），既有代理無法透過 upgradeToAndCall 觸發遷移邏輯。

建議修法：新池直接部署本版本即可，不需要處理本項。既有池若要升級，必須在同一次升級中一併完成：(1) 調升 LOAN_MANAGER_REVISION（LoanManager.sol:29）並新增一支只能在升級交易中執行一次的遷移函式；(2) 遷移函式內以「所有存續 payment 依合約公式計算的應計總和」**即時重算**正確的 accountedInterest 並寫回，不可寫死任何截點金額（升級前只要再發生一次結算，金額就變了）；(3) 發出專用事件（例如 AccountingCorrected(uint256 writtenOff)）供 subgraph 下修 _prevRevenue（該欄位在 mapping 內以 lt 判斷，只會向上調整，鏈上沖銷不會自動反映）；(4) 全程以 upgradeToAndCall 原子完成，避免 [M-9] 描述的初始化空窗。

[L-1] 同一張應收帳款可支撐多筆貸款，合約無「已使用」標記

位置：contracts/LoanManager.sol:173-249

說明：requestLoan／fundLoan（LoanManager.sol:173-249）沒有記錄某張應收帳款憑證是否已經被用來支撐一筆尚未結清的貸款，也沒有在申請時把 NFT 托管起來。同一個 tokenId 因此可以重複提出貸款申請。

影響：第二筆貸款撥款後，資金已離開 pool 並計入 principalOut，但沒有對應的可提領債權 —— 帳務失真而非直接盜取。已確認損失被第二道機制部分擋住：withdrawFunds 要求 safeTransferFrom(msg.sender, address(this), tokenId)（:303），第一筆提領後 NFT 已易主，第二筆無法提領，資金會滯留在 LoanManager 而非落入攻擊者手中。且需要 pool admin 對同一張憑證重複撥款才會發生，故評為 Low。

PoC：已以可執行測試實證
（tests/integration/concrete/loan-manager/poc/AuditFindings.t.sol::test_L1_OneReceivableBacksTwoFundedLoans）：以同一個 tokenId 連續呼叫 requestLoan 兩次都成功（_revertIfInvalidReceivable，LoanManager.sol:964-980，只檢查 buyer／seller／到期日在未來，**不檢查該憑證是否已被使用、也不檢查持有人**），兩筆 fundLoan 也都成功。實測結果：單一 receivable 的 faceAmount 為 100,000,000,000，而 principalOut 為 **200,000,000,000** —— 池子對同一份擔保品放出了兩倍面額的款項。

建議修法：在 requestLoan 檢查該 tokenId 是否已有未結清的貸款（新增 mapping(address => mapping(uint256 => uint16)) activeLoanOfReceivable），或要求申請時即把 NFT 托管給 LoanManager。

已知風險但可接受（B）

ISL-08｜divide-before-multiply

嚴重度：Medium
位置：contracts/LoanManager.sol:782-805

說明：

- LoanManager._queuePayment(uint16,uint256,uint256) (contracts/LoanManager.sol#782-805) performs a multiplication on the result of a division:
  - newRate_ = (_getNetInterest(interest_,feeRate_) * PRECISION) / (dueDate_ - startDate_) (contracts/LoanManager.sol#790)
  - payments[paymentId_] = Loan_Types.PaymentInfo({protocolFee:SafeCast.toUint24(protocolFee_),adminFee:SafeCast.toUint24(adminFee_),startDate:SafeCast.toUint48(startDate_),dueDate:SafeCast.toUint48(dueDate_),incomingNetInterest:SafeCast.toUint128(newRate_ * (dueDate_ - startDate_) / PRECISION),issuanceRate:newRate_}) (contracts/LoanManager.sol#795-802)

判斷依據：屬實但可接受。
newRate_ = _getNetInterest(interest_, feeRate_) * PRECISION / (dueDate_ - startDate_)（LoanManager.sol:790）之中，interest_ 本身已在 _getInterest（:492-501）內做過一次除法，確實有先除後乘。但 PRECISION = 1e27 的放大倍率遠大於任何實際天期，捨入誤差在 wei 級；這也是上游 Maple v2 的原始寫法，經多輪外部審計。上一輪報告在此處指出「真正需要處理的是 _advanceGlobalPaymentAccounting 把系統性重複入帳誤判為捨入誤差」，該缺陷已於 PR #90 修復（:598-605 補回 domainStart 寫回，:624-628 的註解亦已改寫），此處只剩真正的 wei 級捨入。

ISL-16｜reentrancy-no-eth

嚴重度：Medium
位置：contracts/PoolAddressesProvider.sol:166-179

說明：

- Reentrancy in PoolAddressesProvider._updateImpl(bytes32,address,bytes) (contracts/PoolAddressesProvider.sol#166-179):
- External calls:
  - proxy = new TransparentUpgradeableProxy(newAddress,address(this),params) (contracts/PoolAddressesProvider.sol#172)
- State variables written after the call(s):
  - _addresses[id] = proxyAddress = address(proxy) (contracts/PoolAddressesProvider.sol#173)
- 可跨函式重入的狀態變數共 1 個（PoolAddressesProvider._addresses），合計可達函式 7 處；完整清單見掃描原始輸出。

判斷依據：屬實但可接受。_updateImpl（PoolAddressesProvider.sol:166-179）先 new TransparentUpgradeableProxy（:172，建構子會 delegatecall 到 implementation 的 initialize）才寫

_addresses[id]（:173）。重入需要 implementation 在 initialize 期間回呼 provider，而 implementation 是 governor 自己指定的合約，_updateImpl 全部入口皆 onlyGovernor。這屬於「governor 部署惡意 implementation」的既有信任範圍（見 [M-11]），不是額外的攻擊面。建議仍調整為先寫入再部署。

[M-11] 升級與位址改寫權限集中於單一 governor 位址，合約層無多簽或 timelock 要求

位置：contracts/PoolAddressesProvider.sol:145-179

說明：Governable 只維護單一 address governor（Governable.sol:15），採兩段式移轉但無 timelock、無門檻、無第二人核准。該單一位址可執行：(a) setAddress(LOAN_MANAGER, x)（PoolAddressesProvider.sol:145）把 loan manager 換成任意地址；(b) setLoanManagerImpl / setPoolConfiguratorImpl / setWithdrawalManagerImpl / setAddressAsProxy 替換任一核心 implementation；(c) setIsleGlobals 連同 governor 自身一起換掉；(d) 升級 IsleGlobals 與 Receivable（UUPS）。存取控制的**實作是正確的** —— 已逐一確認 onlyGovernor 的比較對象與 IsleGlobals.governor() 一致，無 tx.origin、無恆真條件；問題在於權力邊界本身。

影響：單一私鑰即等同池內全部資金的控制權。以 (a) 為例：Pool 建構子已對 PoolConfigurator 授權 type(uint256).max（Pool.sol:36），而 requestFunds 只檢查 msg.sender == loanManager_（PoolConfigurator.sol:175），因此把 loan manager 指向攻擊者控制的地址後，即可一次提走 pool 全部流動性。合約層沒有任何延遲或第二人核准可供存款人察覺並退出。

PoC：取得 governor 私鑰後，單筆交易呼叫 PoolAddressesProvider.setAddress(LOAN_MANAGER, attacker)，接著以該地址呼叫 PoolConfigurator.requestFunds(pool.totalAssets()) 即提走全部流動性。這正是 audit/DOMAIN_RESEARCH.md 的 D-RWA-02（來源：rwa.md，last_reviewed 2026-08-12）所記載的模式：2025 年 RWA 領域最大宗事故即為單一 deployer 私鑰被取得後呼叫 upgradeToAndCall 替換 implementation，損失約 850 萬美元。

接受理由：此為 audit/DOMAIN_RESEARCH.md 的 D-RWA-02（來源：rwa.md，last_reviewed 2026-08-12），該條記載 2025 年 RWA 領域最大宗事故即為單一 deployer 私鑰被取得後呼叫 upgradeToAndCall 替換 implementation，損失約 850 萬美元。本專案的存取控制**實作正確**（已逐一確認 onlyGovernor 的比較對象與 IsleGlobals.governor() 一致，無 tx.origin、無恆真條件），問題在於權力邊界本身。列為 B 而非 A，因為這是明確的產品設計選擇而非程式缺陷 —— 但緩解措施屬於部署與營運層，必須在鏈下落實並向存款人揭露。

建議修法：短期（營運層，不需改合約）：將 governor 位址移轉至多簽錢包（建議 3-of-5 以上，簽署人分散於不同組織與金鑰保管方式），並把此安排連同簽署人組成向存款人公開揭露。長期（合約層）：在 Governable 加入 timelock —— 特權操作分為提案與執行兩段，中間留足夠的公告期讓存款人有時間退出；並對 setAddress(LOAN_MANAGER, ...) 這類可直接導致資金外流的操作設定較長的延遲。

[I-1] 無儲備證明機制：流通份額對應的債權無法被外部獨立驗證

位置：contracts/Receivable.sol:59-92

說明：系統沒有任何機制可讓外部獨立驗證「流通份額所對應的債權真實存在且金額正確」。Receivable 的憑證可由任何人鑄造（見 [M-6]），應計利息由合約自行累加而無對照來源，資產淨值的正確性完全依賴合約內

部帳務自身正確。對照 audit/DOMAIN_RESEARCH.md 的 D-RWA-05（來源：rwa.md）四個標準查證問題，四項皆為否。

影響：帳務一旦出錯，系統本身不會發現，也沒有任何外部方能在不取得內部資料的情況下察覺。本次 postmortem 正是這個缺口的實證：accountedInterest 累積了 $3,281 沒有任何真實債權對應，是靠人工重放鏈上事件、與獨立重算的應計總和比對才查出來的。此缺口決定的不是「會不會出錯」，而是「下一次帳對不上時要多久才會現形」。

PoC：非可被主動利用的漏洞，而是機制缺失。其存在已由本次事件實證：ISL-105 造成的重複入帳在鏈上持續累積且被計入份額價格，期間沒有任何合約層機制、事件或對外介面會揭露帳實不符；發現途徑是人工重放事件並獨立重算應計總和。

接受理由：對照 audit/DOMAIN_RESEARCH.md 的 D-RWA-05（來源：rwa.md）四個標準查證問題，四項皆為否。本次 postmortem 正是這個缺口的實證：accountedInterest 累積了 $3,281 沒有任何真實債權對應，而系統本身沒有任何機制會發現 —— 是靠人工重放鏈上事件、與獨立重算的應計總和比對才查出來的。列為 Informational 是因為這是機制缺失而非可被直接利用的漏洞，但它決定了「下一次帳對不上時要多久才會現形」。

建議修法：短期：對外發布可獨立驗證的儲備報告 —— 定期公布全部存續 payment 的清單與應計總和，讓外部可用鏈上事件自行重算並與 assetsUnderManagement() 比對。中期：在合約層加入不變量檢查，例如於結算路徑斷言 accountedInterest 不超過所有存續 payment 的理論上限，違反時發出事件而非靜默累積。長期：引入 attestation 機制，使 Receivable 憑證對應的真實發票可被第三方背書驗證（與 [M-6] 的長期建議同一方向）。

附錄：發現處置分類

本報告對每一筆發現標示兩個獨立欄位：**嚴重度**（Critical／High／Medium／Low／Informational，由工程團隊依實際影響判定）與**處置**（下列 A／B／C／D）。掃描工具自身回報的 impact 若與本報告呈現的嚴重度不同，該筆會並列印出兩者與調整理由。

- **A**｜**已確認需修復**：確認為真實問題（多涉及資金流向或權限控制邏輯），必須修復。
- **B**｜**已知風險但可接受**：問題確實存在，但經工程團隊評估風險可控（例如僅管理者可呼叫、另有其他層級防護），附具體理由後接受並於報告揭露。
- **C**｜**誤報**：靜態分析限制造成的誤判，實際已有防護機制或該判斷邏輯不適用。
- **D**｜**待確認**：尚無法判定歸屬者一律列此類；判讀信心不足時寧列 D，不猜測分類。此類項目均附「要確認什麼／由誰確認／兩種答案各自的處置」。